Строки

Алгоритмы со строками

В языке C# строковые значения представляет тип **string**, а вся функциональность работы с данным типом сосредоточена в классе **System.String**. Собственно string является псевдонимом для класса String. Объекты этого класса представляют текст как последовательность символов Unicode. Максимальный размер объекта String может составлять в памяти 2 ГБ, или около 1 миллиарда символов.

```
string s1 = "hello";
string s2 = new String('a', 6); // результатом будет строка "aaaaaa"
string s3 = new String(new char[] { 'w', 'o', 'r', 'l', 'd' });
string s4 = new String(new char[] { 'w', 'o', 'r', 'l', 'd' }, 1, 3); // orl
```

Перебор строк

```
string message = "hello";

for(var i =0; i < message.Length; i++)
{
    Console.WriteLine(message[i]);
}
foreach(var ch in message)
{
    Console.WriteLine(ch);
}
```

Разделение строк

```
string text = "И поэтому все так произошло";

string[] words = text.Split(new char[] { ' ' });

foreach (string s in words)
{
    Console.WriteLine(s);
}
```

Поиск в строке

```
string s1 = "hello world";
char ch = 'o';
int indexOfChar = s1.IndexOf(ch); // равно 4
Console.WriteLine(indexOfChar);

string substring = "wor";
int indexOfSubstring = s1.IndexOf(substring); // равно 6
Console.WriteLine(indexOfSubstring);
```

Задача.
Числа и строки

- Дана строка, содержащая слова и числа. Нужно найти все числа, сложить их и вернуть сумму.

Пример: "abc123def45ghi6" -> 21(1+2+3+4+5+6)

Используйте функцию IsDigit для проверки десятичного числа в строке

```
string input = "abc123def45ghi6";
int sum = 0;

foreach (char c in input)
{
    if (char.IsDigit(c))
        sum += int.Parse(c.ToString());
}
Console.WriteLine(sum);
```

Задача на уникальность

- Определите, содержит ли строка все уникальные символы, не используя дополнительные структуры данных.

"abcdef" -> true

"hello" -> false

палиндром

это число или строка, которая читается одинаково слева направо и справа налево. Например: 121 или «racecar».

Напишите программу функцию, которая принимает строку и возвразает
true- если строка палиндром
false-если нет

```
bool IsPalindrome(string str)
{
    int i = 0, j = str.Length - 1;
    while (i < j)
    {
        if (str[i++] != str[j--]) return false;
    }
    return true;
}


Console.WriteLine(IsPalindrome("AAAbAAA"));
```

КМП

Алгоритм Кнута - Морриса - Пратта (КМП) - популярный и эффективный метод поиска меньшей строки (называемой "шаблоном") внутри большей строки (называемой "текстом"). Он ускоряет процесс поиска, избегая ненужных сравнений за счет предварительного вычисления . Алгоритм КМП состоит из двух основных этапов:

Предварительная обработка шаблона: перед поиском KMP создает массив (так называемый массив LPS) на основе шаблона, который помогает определить, какая часть шаблона уже совпала.

Поиск в тексте: используя массив LPS, KMP быстро пропускает части текста , в которых шаблон точно не может совпасть, что повышает эффективность поиска.

КМП.Общее

Главная идея: при несовпадении символов мы не откатываем указатель в тексте назад, а используем информацию о предыдущих совпадениях.

- 1. формирование массива т, используемого при сдвиге образа вдоль строки;
- 2. поиск образа в строке.

Префикс – символы в начале образа

Суфикс – символы в конеце образа

String:	a	b	a	b	c	a	b	c	a	b	a	b	a	b	d
	i														
Pattern:	a	b	a	b	d										
LPS:	0	0	1	2	0										
	j														

массив LPS

Вот как мы создаем массив LPS для шаблона P = "ABABCABAB":

Индекс (i)	0	1	2	3	4	5	6	7	8
Узор	A	B	A	B	C	A	B	A	B
Значение LPS	0	0	1	2	0	1	2	3	4

Псевдокод этап 1

$\pi[0]=0;$

$j=0;$

$i=1;$

пока не закончился образ проверяем одно единственное условие:

if $a_i == a_j$ then $\pi[i] = j+1$; i++; j++; // $a_i == a_j$

else if $j == 0$ then $\pi[i]=0$; i++; // $a_i != a_j$ и $j == 0$

else $j = \pi[j-1]$; // $a_i != a_j$ и $j != 0$

Псевдокод 2го этапа

$k = 0;$

$l = 0;$

пока не закончилась строка проверяем одно единственное условие:

if $t_k == a_l$ then k++; l++; if $l == n$ then образ найден

else if $l == 0$ then k++; if $k == m$ then образа в стр.нет

else $l = \pi[l - 1]$;

/* первая строка условия: $t_k = a_l$

вторая строка: $t_k \neq a_l$ и $l = 0$

третья строка: $t_k \neq a_l$ и $l \neq 0$ */

Сопоставление шаблона с текстом

Теперь с помощью массива LPS мы сравниваем шаблон с текстом:

•Начните с начала текста и шаблона.

•Сравнивайте символы один за другим.

•Если они совпадают, перемещайте оба указателя (один в тексте, другой в шаблоне) вперед.

•Если они не совпадают, используйте массив LPS, чтобы определить, на какое расстояние нужно сдвинуться в шаблоне, не перепроверяя уже просмотренные части текста.

КМП.Пример

Текст: A B A B A B A C
Образец: A B A B A C

ЭТАП 1:

π[0] = 0 // "A"
π[1] = 0 // "AB"
π[2] = 1 // "ABA" (префикс "A" = суффикс "A")
π[3] = 2 // "ABAB" (префикс "AB" = суффикс "AB")
π[4] = 3 // "ABABA" (префикс "ABA" = суффикс "ABA")
π[5] = 0 // "ABABAC"

π = [0, 0, 1, 2, 3, 0]

ЭТАП 2:

i=0, j=0: 'A' == 'A' → j=1
i=1, j=1: 'B' == 'B' → j=2
i=2, j=2: 'A' == 'A' → j=3
i=3, j=3: 'B' == 'B' → j=4
i=4, j=4: 'A' == 'A' → j=5
i=5, j=5: 'B' != 'C' → j = π[4] = 3

i=5, j=3: 'B' != 'A' → j = π[2] = 1
i=5, j=1: 'B' == 'B' → j=2

i=6, j=2: 'A' == 'A' → j=3
i=7, j=3: 'C' == 'C' → j=4
i=8, j=4: j==6? Нет, i=8 вышли за границу

Текст: A B A B A B A C
Образец: A B A B A C
↑
Позиция 2 (индексация с 0)

Образец "ABABAC" найден в тексте один раз на позиции 2